

Python

Functions to Exception Handling

Hour 2 • Hour 3 • Hour 4

▶ Functions & Scope

▶ Lambda Expressions

▶ Map & Filter

▶ Nested Functions

▶ File Handling

▶ Exception Handling

RA: Muhammad Huzifa

Topic 1: Functions

Hour 2 — Understanding reusable blocks of code

What is a Function?

- A function is a named block of instructions that performs one specific task
- Think of it like a recipe — you write it once and use it as many times as you like
- You call the function by writing its name followed by parentheses
- A function can receive inputs (called parameters) to work with
- A function can send back a result (called a return value) to the caller
- Functions help us avoid writing the same code over and over again

Daily Life Analogy:

A washing machine is like a function. You put in dirty clothes (input), press start (call), and get clean clothes (output). You don't need to know how the machine works inside!

Why Do We Need Functions?

Avoid Repetition

Instead of writing the same calculation 10 times, write a function once and call it 10 times.

Easier to Fix

If there is a mistake, you only fix it in one place — inside the function.

Readable Code

Code with functions reads like plain English:
`calculate_bill()`,
`check_attendance()`.

Reuse Across Projects

A function you write today can be used in tomorrow's project.

- Functions break a big problem into small, manageable pieces
- Each function should do ONE thing only — this is the golden rule
- Well-named functions make code self-documenting

The Four Parts of a Function

1

def keyword

Tells Python you are about to define a function

2

Function Name

A clear name that describes what the function does, e.g. `calculate_bill`

3

Parameters

Inputs inside the parentheses — the information the function needs to do its job

4

return statement

Optional — sends a result back to whoever called the function

Functions in Daily Life

Calculator

A function called `add_numbers` takes two numbers and gives back their sum. Like pressing `+` on a calculator.

Student Grade

A function called `get_grade` takes a student's marks and tells you if they passed or failed.

Shopping Bill

A function called `calculate_bill` takes item price and quantity and returns the total amount.

Temperature

A function called `convert_to_fahrenheit` takes a temperature in Celsius and returns the Fahrenheit value.

Attendance

A function called `mark_attendance` records a student's name and increments the count.

Common Mistakes — Functions



Forgetting the colon (:)

Every function definition must end with a colon. Without it Python gives a syntax error.



No indentation in the body

All lines inside a function must be indented (4 spaces). Forgetting this breaks the function.



Calling before defining

You must define a function before you call it. Writing the call first causes a `NameError`.



Forgetting to use return

If you need the result back, you must write `return`. Just printing inside is not the same.



Giving confusing names

A function named `do_stuff` is unclear. Use descriptive names: `calculate_total`, `check_age`.

Interview-Style Questions — Functions

Q1

What is a function and why do we use them in programming?

Q2

What is the difference between a parameter and an argument?

Q3

What does the return keyword do? What happens if you forget it?

Q4

Can a function work without any parameters? Give an example.

Q5

Why is it important to give functions clear and descriptive names?

Topic 2: Variable Scope

Understanding where your variables live and who can see them

What is Variable Scope?

- Scope means: where in your program can a variable be seen and used?
- A variable created inside a function is LOCAL — only that function can use it
- A variable created outside any function is GLOBAL — all parts of the program can use it
- When a function finishes, its local variables are deleted from memory
- Two variables with the same name can exist — one local and one global — without conflict

Analogy:

Think of a school building. A global variable is like a notice on the main board — everyone sees it. A local variable is a note on your desk — only you can read it, and it disappears when you leave class.

Local vs Global — Side by Side

LOCAL Variable

- Defined inside a function
- Only that function can see it
- Deleted when the function ends
- Cannot be used outside the function
- Safe from accidental changes elsewhere

GLOBAL Variable

- Defined outside all functions
- Any part of the code can read it
- Exists for the entire program's life
- Must use global keyword to modify it
- Changing it affects the whole program

The global Keyword

- By default, functions can only READ global variables, not change them
- If you want to CHANGE a global variable inside a function, write: `global variable_name`
- Without the global keyword, Python creates a new local variable with the same name instead
- Real-life use case: tracking attendance count, bank balance, or total sales
- Use global sparingly — overusing it makes programs hard to understand

Example:

Attendance counter starts at 0. Every time `mark_attendance()` is called, the counter goes up by 1. Without `global`, the counter would reset to 0 every time the function is called.

Common Mistakes — Variable Scope



Using a local variable outside its function

Python will give a `NameError` because the variable only exists inside the function.



Forgetting the `global` keyword

You modify the variable inside but nothing changes outside — very confusing to debug.



Naming local and global variables the same

This is allowed but very confusing. Use different names to avoid mistakes.



Over-relying on global variables

Too many global variables make code hard to track and test. Pass values as parameters instead.



Assuming all variables are global

Always check where a variable is defined before using it in a function.

Interview-Style Questions — Scope

Q1

What is the difference between a local and a global variable?

Q2

What happens if you try to use a local variable outside its function?

Q3

When do you need to use the global keyword?

Q4

Can two variables have the same name — one local and one global? What happens?

Q5

Why is it generally better to pass values as parameters than use global variables?



Topic 3: Lambda Expressions

Writing quick, one-line functions without a name

What is a Lambda Expression?

- A lambda is a very short, nameless function written in a single line
- It is perfect for small, quick calculations that you only need once
- A lambda takes inputs, performs one operation, and immediately gives back the result
- You can store a lambda in a variable and call it just like a regular function
- Regular functions use `def`; lambda functions use the word `lambda`
- Lambdas cannot contain multiple steps — they are designed for simple tasks only

Analogy:

A lambda is like a sticky note reminder. A full function is a proper document. For a quick note — 'double the price' — a sticky note is perfect. For a full report, you need a proper document.

When to Use Lambda — Daily Life Examples

Temperature

Convert Celsius to Fahrenheit in one step — a perfect one-line calculation with no extra logic needed.

Tax Calculator

Add 17% sales tax to any price — one simple formula that never changes.

Discount

Calculate a discounted price: take the original price and the percentage, return the final price.

Package Weight

Convert weight from kilograms to grams — multiply by 1000. Simple, one-step, perfect for lambda.

Sorting Students

Sort a list of students by their marks from highest to lowest using lambda as the sorting key.

Common Mistakes — Lambda Expressions



Using lambda for complex logic

If your function needs if/else on multiple lines or loops, use `def` — not `lambda`.



Forgetting it returns automatically

You do NOT write `return` in a `lambda`. The expression after the colon is automatically returned.



Using confusing one-letter names

`lambda x: x*2` is fine for simple cases, but name your variables clearly when possible.



Thinking lambda replaces def

`Lambda` is a shortcut, not a replacement. Most real functions should still use `def`.



Not storing the lambda in a variable

If you create a `lambda` but do not store it, you cannot use it again.

Interview-Style Questions — Lambda

Q1 What is a lambda expression and how is it different from a regular function?

Q2 Can a lambda function have more than one line of code? Why or why not?

Q3 Why would you use lambda with `sorted()` when sorting a list of students by marks?

Q4 Do you need to write `return` inside a lambda? Explain.

Q5 Give one real-life example where a lambda expression would be useful.

Topic 4: Map and Filter

Processing entire lists in one clean step

What is map()?

- `map()` applies a function to EVERY item in a list and gives you a new list of results
- You give it two things: a function (what to do) and a list (what to do it to)
- The result is a map object — wrap it with `list()` to see the results
- You do NOT need to write a loop — `map()` handles every item automatically
- Imagine stamping every paper in a pile with the same stamp — that is `map()`

Daily Life Example:

You have a list of prices in Rupees. You want every price converted to Dollars. Instead of converting one by one, you use `map()` to convert ALL prices at once with a single instruction.

What is filter()?

- `filter()` goes through a list and KEEPS only the items where a condition is True
- You give it a function that returns True or False, and a list
- Items where the function returns True are kept; the rest are removed
- Like a sieve — only what passes through the holes is kept
- Use `filter()` when you want to select a subset from a larger collection

Daily Life Example:

You have a list of student marks. You want only the students who passed (marks ≥ 50). `filter()` automatically goes through all marks and gives back only the passing ones.

map() vs filter() — Key Difference

map()

- TRANSFORMS every item
- Same number of items in and out
- Applies a function to change each item
- Input list size = output list size
- Example: convert all prices to dollars

filter()

- SELECTS some items
- Output may have fewer items
- Applies a True/False test to each item
- Input list size $\geq$ output list size
- Example: keep only students who passed

Real-Life Uses of map() and filter()

Price Increase

A shop raises all prices by 10%. `map()` applies the increase to every price in the list at once.

Pass/Fail Filter

From a class of 40 students, `filter()` extracts only those with marks above 50.

Bulk Conversion

Converting a whole month of daily temperatures from Celsius to Fahrenheit using `map()`.

Age Check

From a list of cinema visitors, `filter()` selects only those aged 18 and above.

Bill Totals

A shopping app uses `map()` to calculate the total for each item (price $\times$ quantity).

Grade Assignment

`map()` assigns a grade letter to every student based on their marks.

Common Mistakes — Map and Filter



Forgetting to wrap with list()

map() and filter() return objects, not lists. Always wrap with list() to see the results.



Confusing map and filter

Use map to change/transform. Use filter to select/remove. Do not mix them up.



Passing the wrong argument order

The function always comes first, then the list. Swapping them causes a TypeError.



Using filter when you need map

If every item needs a change, use map. If you need to remove some items, use filter.



Expecting original list to change

map() and filter() create NEW lists. The original list is untouched.

Interview-Style Questions — Map & Filter

Q1

What does `map()` do? How is it different from writing a for loop?

Q2

What does `filter()` return — the same list, a smaller list, or a larger list? Why?

Q3

If you want to change every element in a list, which do you use: `map` or `filter`?

Q4

Why do you need to wrap `map()` or `filter()` with `list()` to see the results?

Q5

Give a real-life example where you would use `map()` and one where you would use `filter()`.

Topic 5: Nested Functions

Functions living inside other functions

What are Nested Functions?

- A nested function is simply a function defined INSIDE another function
- The inner function can only be called from within the outer function
- The inner function can access variables from its outer function
- Useful when a helper task belongs only to one function and nowhere else
- It keeps related code together and hides small helpers from the rest of the program

Analogy:

A restaurant kitchen has a head chef (outer function) and a sous-chef (inner function). The sous-chef only works for that head chef — nobody outside the kitchen calls the sous-chef directly.

Why Use Nested Functions?

Encapsulation

Hide helper logic inside the main function. Other parts of your program cannot accidentally call the inner function.

Better Organisation

Group related logic together. The outer function is the main task; inner functions are its sub-tasks.

Cleaner Code

Instead of creating many separate functions at the top level, keep private helpers inside the function that uses them.

Reusing Local Logic

If the same small calculation is needed several times inside one function, put it in an inner function.

Nested Functions in Daily Life

- A shopping bill function contains inner functions: one to calculate subtotal, another to add tax
- A student report card function contains inner functions: one to average marks, one to assign grades
- A password validator function contains inner functions: one to check length, one for uppercase, one for digits
- An ATM function contains inner functions: one to validate PIN, one to check balance, one to dispense
- The outer function is the main process; inner functions are its step-by-step helpers

Common Mistakes — Nested Functions



Calling the inner function from outside

The inner function is private. Trying to call it from outside the outer function causes a `NameError`.



Overusing nested functions

Not every function needs inner functions. Only use them when the helper is genuinely private and local.



Confusing with recursion

Nested functions call a different function. Recursion is when a function calls ITSELF. They are different.



Forgetting to call the inner function

You define the inner function but forget to call it inside the outer function — so nothing happens.



Making inner functions too complex

Inner functions should be small and simple. If they grow too large, promote them to proper top-level functions.

Interview-Style Questions — Nested Functions

- Q1** What is a nested function? Give a simple real-life example.
- Q2** Can the inner function be called from outside the outer function? Why not?
- Q3** What is the benefit of using a nested function over writing a separate function?
- Q4** How is a nested function different from a recursive function?
- Q5** When would you choose NOT to use a nested function?

Topic 6: File Handling

Hour 4 — Reading and writing data that lasts beyond the program

What is File Handling?

- File handling means writing information to a file and reading it back later
- When your program ends, all variables disappear — but files on disk stay forever
- Python lets you create, write, read, and add to text files with simple commands
- You open a file, perform your operations, then close it when done
- The with statement handles opening and closing automatically — always use it

Analogy:

A program's memory is like a whiteboard — it is erased when you leave. A file is like a notebook — what you write stays there even after you close it and come back tomorrow.

Three File Modes — What They Do

"w" — Write

- Opens the file for writing
- Creates the file if it does not exist
- ERASES all existing content first
- Use when you want to start fresh

"r" — Read

- Opens the file for reading only
- File must already exist
- Does not change the file
- Use when you want to see saved data

"a" — Append

- Opens the file for adding new content
- Creates the file if it does not exist
- Does NOT erase existing data
- Use when you want to add more records

Why Use with open()?

- The with statement automatically closes the file when you are done — even if an error occurs
- Forgetting to close a file can corrupt data or lock the file for other programs
- with open() is the professional and safe way to handle files in Python
- The file is only open inside the with block — it closes when the block ends
- Always prefer with open() over manually calling open() and close()

Think of with as an automatic door closer:

You walk in (open the file), do your work, and the door closes behind you automatically (file is closed). You never have to remember to close it yourself.

Real-Life Uses of File Handling

Student Records

Save student names and marks to a file so they are available next time the program runs.

Attendance Register

Append each student's attendance daily so a full record builds up over the school year.

Bank Transactions

Save every deposit and withdrawal so the bank has a permanent record of all activity.

Shopping History

Store each purchase so customers can review their order history later.

Exam Results

Write each student's grade to a file that the principal can open and print at any time.

Daily Notes

A diary app that saves what you write each day so it is there when you come back tomorrow.

Common Mistakes — File Handling



Using 'w' mode to add new data

Mode 'w' erases existing content. To add without erasing, always use 'a' (append) mode.



Forgetting to close the file

Without with open(), you must call file.close(). Forgetting this can corrupt the file.



Reading a file that does not exist

If the file is missing, Python raises a FileNotFoundError. Always check or handle this error.



Forgetting to strip newlines when reading

Each line read from a file includes a hidden newline character. Use .strip() to remove it.



Assuming file contents are numbers

Everything read from a file is a string. Convert with int() or float() before doing maths.

Interview-Style Questions — File Handling

Q1

What is the difference between write mode 'w' and append mode 'a'?

Q2

Why should you always use `with open()` instead of just `open()`?

Q3

What happens if you try to read a file that does not exist?

Q4

If you read numbers from a file, can you immediately use them in maths? Why or why not?

Q5

Why do we need to save data to files instead of just keeping it in variables?

Topic 7: Exception Handling

Hour 4 — Writing programs that survive real-world errors

What is an Exception?

- An exception is an error that occurs while your program is running
- When Python hits an error it cannot handle, it STOPS the program immediately
- Common exceptions: dividing by zero, opening a missing file, typing letters instead of numbers
- Without exception handling, one small mistake crashes the entire program
- Exception handling lets your program deal with errors gracefully and continue running

Analogy:

An ATM does not shut down if you enter the wrong PIN. It shows a message and asks you to try again. That is exception handling — the program survives the mistake and gives a helpful response.

The try / except Structure

try

Put the code that MIGHT cause an error here. Python tries to run it.

except

If an error occurs in try, Python jumps here and runs this block instead of crashing.

else

Optional — runs ONLY if no error occurred. The success path.

finally

Optional — ALWAYS runs, whether there was an error or not. Good for cleanup.

Common Exception Types

ZeroDivisionError

Dividing a number by zero — like calculating cost per person when nobody is there.

ValueError

Wrong type of value — like entering 'twenty' when a number is expected.

FileNotFoundError

Trying to open a file that does not exist yet.

TypeError

Using wrong data types — like trying to add a word to a number.

IndexError

Accessing position 5 in a list that only has 3 items.

KeyError

Looking up a name in a dictionary that is not there.

Real-Life Uses of Exception Handling

- ATM machine — if the user types letters instead of numbers, show an error message, do not crash
- Age checker — if the user enters their name instead of their age, ask again politely
- File reader — if the student file is missing, show 'file not found' instead of crashing
- Calculator — if the user tries to divide by zero, show a message instead of stopping
- Shopping bill — if a price is missing or wrong type, handle it and continue with the rest
- Password validator — if no input is given, catch the error and prompt again

The finally Block — Always Runs

- finally always executes — whether there was an error or not
- Use it for cleanup tasks that must happen regardless of success or failure
- Common uses: closing a file, printing a confirmation message, resetting a counter
- Even if an error occurs and except runs, finally still runs afterwards
- It is like a goodbye that always happens when leaving a building, no matter what

Example:

A file reading function: whether the file is found or not, finally always prints 'File operation completed.' This confirms to the user that Python attempted the task.

Common Mistakes — Exception Handling

X

Catching all errors with bare except

Writing just 'except' catches EVERYTHING including keyboard interrupts. Always specify the error type.

X

Ignoring the error silently

An empty except block swallows errors without telling anyone. Always print a helpful message.

X

Putting too much code in try

Only put the risky line in the try block, not your whole program. This makes debugging easier.

X

Thinking except replaces fixing bugs

Exception handling is for unavoidable runtime errors — not for hiding bugs in your logic.

X

Forgetting ValueError for user input

Whenever you convert user input with `int()` or `float()`, always wrap it in `try/except ValueError`.

Interview-Style Questions — Exception Handling

Q1 What is an exception? Give a real-life example of when one might occur.

Q2 What is the difference between try and except? What happens in each block?

Q3 What does the finally block do and when would you use it?

Q4 Why is it a bad idea to use a bare except that catches every possible error?

Q5 What exception would occur if a user types their name when a number is expected?

Session Summary — All Topics Covered

1

Functions

Define reusable blocks of code; pass inputs, receive outputs

2

Variable Scope

Local variables live inside functions; global variables live outside

3

Lambda Expressions

One-line nameless functions for quick, simple calculations

4

Map & Filter

Apply a function to all items (map) or select items (filter) from a list

5

Nested Functions

Define helper functions inside a main function for organisation

6

File Handling

Write and read data that persists beyond the program using files

7

Exception Handling

Catch runtime errors gracefully so the program does not crash

Rapid Revision — Quick-Fire Questions

1

What keyword do you use to define a function in Python?

2

What is a local variable? Can it be used outside its function?

3

Write a lambda that multiplies a number by 3.

4

What is the difference between `map()` and `filter()`?

5

What does 'a' mode do when opening a file?

6

Which exception is raised when dividing by zero?

7

What does the `finally` block guarantee?

8

Can a function have no return statement? What does it return by default?

Thank You!

Keep practising. Keep building.

- ✓ Functions
- ✓ Variable Scope
- ✓ Lambda
- ✓ Map & Filter
- ✓ Nested Functions
- ✓ File Handling
- ✓ Exception Handling

RA: Muhammad Huzifa